C PROGRAMMING • STUDY GUIDE

Character Strings in C

A complete, exam-oriented reference covering string fundamentals, the C standard string library, and the string–array relationship — built for university exams, competitive tests, and teacher recruitment exams.

7	10+	15+	1
Chapters	Diagrams	Solved Programs	Page Revision Sheet

```c
#include <stdio.h>
#include <string.h>

int main() {
    char name[20] = "Language";
    // name[8] = '\0'  --> end marker
    printf("Length = %d", strlen(name));
    return 0;
}
```

Topics: What is a String | strlen(), strcpy(), strcat(), strcmp() | String vs 1-D Char Array

Table of Contents

1. Introduction to Character Strings 3

Definition, null character, characteristics, memory diagram

2. Declaring and Initializing Strings 5

Declaration syntax, initialization, input/output

3. Standard Library String Functions 7

strlen(), strcpy(), strcat(), strcmp(), strn-variants

4. Similarity Between String and 1-D Array of Char 12

Relationship, similarities/differences, comparison table

5. Programming Examples 14

Length, copy, concatenate, compare, manipulation programs

6. Common Errors and Best Practices 16

Missing null terminator, buffer overflow, safe coding

7. Quick Revision Section 18

Definitions, syntax table, FAQs, one-page notes

> **How to use this guide**
>
> Read Chapter 1 for concepts, Chapters 2–3 for declaring strings and mastering the library functions, Chapter 4 to see how strings relate to character arrays, Chapter 5 for practice, and keep Chapter 7 handy the night before your exam for rapid revision.

CH 1 Introduction to Character Strings

1.1 What is a String?

A **string** is a sequence of characters stored contiguously in memory and terminated by a special character called the **null character**, written as `'\0'` (ASCII value 0). In C, there is no separate built-in "string" data type — a string is simply implemented as a **one-dimensional array of characters** whose last valid element is followed by `'\0'`.

Because of this, a string in C is often called a **null-terminated character array**. The null character itself is not printed or counted as part of the visible text; it only acts as a marker that tells library functions "the string ends here."

1.2 Strings in C Programming

- A string constant such as `"HELLO"` is stored by the compiler as `H E L L O \0` — 6 bytes, not 5.
- String variables are declared using `char` arrays, e.g. `char city[10];`
- C provides the header file `<string.h>` containing ready-made functions to manipulate strings.
- Strings can be read using `scanf()`, `gets()` (unsafe, avoid), or `fgets()`, and displayed using `printf()` or `puts()`.

1.3 The Null Character `'\0'`

> **Why the null character matters**
>
> Every string-handling function (`strlen`, `strcpy`, `strcat`, `strcmp`, `printf("%s")` …) scans memory **one byte at a time until it finds `'\0'`**. If the null character is missing or overwritten, the function keeps reading into unrelated memory, producing garbage output or a crash.

Property	Detail
Character	`'\0'` (also written `NUL`)
ASCII value	0
Size	1 byte
Role	Marks the logical end of a string inside a character array

The null character is added automatically for string literals like `"abc"` and by `scanf("%s")`, but not if you fill a char array element-by-element yourself.

1.4 Characteristics of Strings

- Stored as a contiguous block of memory.
- Always terminated by `'\0'`.
- Array size must be at least (number of characters + 1).
- Name of the array acts as a pointer to its first character.
- Individual characters are accessed using indexing, e.g. `str[0]`.
- C strings are mutable if stored in a writable array.
- String literals used directly (e.g. `"Hi"`) may be stored in read-only memory.

1.5 Applications of Strings

Strings are used for storing names, addresses, and text data; building user interfaces and menus; file names and paths; parsing input/commands; text processing and pattern matching; and forming the basis of more advanced data structures such as tries and string-based hash tables.

1.6 Diagram: Internal Representation of a String in Memory

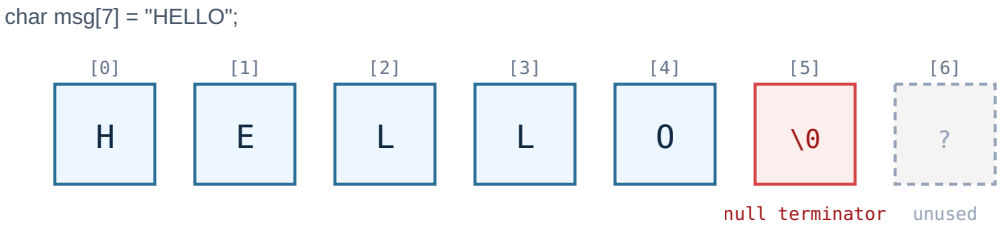


Fig 1.1 — The string "HELLO" stored in a 7-byte character array. Index 5 holds '\0'; index 6 is unused/garbage.

Chapter 1 Summary

- A string = array of `char` + a terminating `'\0'`.
- C has no separate string type; it is built on top of character arrays.
- The null character is essential — its absence causes undefined behaviour.

CH 2 Declaring and Initializing Strings

2.1 Declaration Syntax

```
SYNTAX
char variable_name[size];
```

Example: `char name[20];` declares an array capable of holding a string up to 19 characters long (1 byte reserved for `'\0'`).

2.2 Initialization Methods

Method	Example	Notes
String literal	`char s[6] = "Hello";`	'\0' added automatically
Without size	`char s[] = "Hello";`	Compiler computes size = 6
Character-by-character	`char s[6] = {'H','e','l','l','o','\0'};`	Must add `'\0'` manually
Pointer to literal	`char *s = "Hello";`	Points to read-only string constant

Common pitfall

`char s[5] = "Hello";` is **invalid/unsafe** — "Hello" needs 6 bytes (5 letters + '\0') but only 5 are provided, leaving no room for the terminator.

2.3 String Input and Output

Operation	Function	Example	Remark
Input (single word)	`scanf("%s", str)`	`scanf("%s", name);`	Stops at whitespace; no `&` needed for array
Input (whole line)	`fgets()`	`fgets(name, 20, stdin);`	Safe — takes a size limit; may include '\n'
Output	`printf("%s", str)`	`printf("%s", name);`	Prints till '\0'
Output (line)	`puts(str)`	`puts(name);`	Adds newline automatically

2.4 Examples with Output

2.4.1 Reading a single word — scanf("%s")

```c
#include <stdio.h>

int main() {
    char name[20];
    printf("Enter your name: ");
    scanf("%s", name);
    printf("Hello, %s!\n", name);
    return 0;
}
```

SAMPLE RUN

```
Enter your name: Aditi
Hello, Aditi!
```

`scanf("%s")` reads only up to the first whitespace — so entering `Aditi Sharma` would store just `"Aditi"` in `name`.

2.4.2 Reading a whole line — fgets()

```c
#include <stdio.h>

int main() {
    char sentence[30];
    printf("Enter a sentence: ");
    fgets(sentence, 30, stdin);
    printf("You typed: %s", sentence);
    return 0;
}
```

SAMPLE RUN

```
Enter a sentence: Aditi Sharma
You typed: Aditi Sharma
```

`fgets()` reads the entire line (including spaces) up to 29 characters, and stops safely at the buffer limit even if the input is longer — unlike `scanf("%s")`.

2.4.3 Displaying a string — printf("%s") vs puts()

```c
#include <stdio.h>

int main() {
    char city[] = "Mumbai";
    printf("%s", city);   // no newline added
    printf("\n");
    puts(city);              // newline added automatically
    return 0;
}
```

OUTPUT

```
Mumbai
Mumbai
```

Both lines print "Mumbai", but `puts()` appends a newline (`'\n'`) automatically, while `printf("%s")` does not unless you add one explicitly.

Chapter 2 Summary

- Always allocate `(max characters + 1)` bytes for a char array holding a string.
- `scanf("%s")` reads a single token; `fgets()` reads a full line safely.
- `printf("%s")` / `puts()` display characters until `'\0'` is encountered.

CH 3 Standard Library String Functions

All functions below are declared in the header file `<string.h>` and must be included before use.

3.1 strlen() — String Length

strlen() Returns the number of characters in a string, excluding '\0'

SYNTAX

```
size_t strlen(const char *str);
```

Item	Detail
Parameter	`str` — pointer to a null-terminated character array
Return value	`size_t` (unsigned integer) — count of characters before `'\0'`
Working	Starts a counter at 0, moves through memory one byte at a time, incrementing the counter until `'\0'` is found

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Computer";
    printf("Length = %lu", strlen(str));
    return 0;
}
```

OUTPUT

```
Length = 8
```

3.1.1 Diagram — How strlen() Works

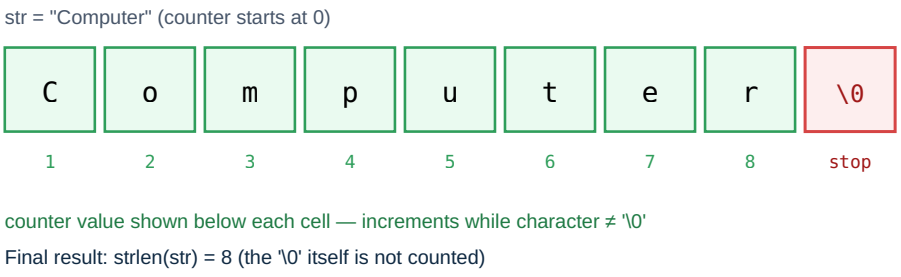


Fig 3.1 — strlen() walks the array counting characters and stops the instant it meets '\0'.

3.2 strcpy() — String Copy

strcpy() Copies the contents of one string into another

SYNTAX

```
char *strcpy(char *dest, const char *src);
```

Item	Detail
Parameters	`dest` — destination array (must be large enough); `src` — source string (unchanged)
Return value	Pointer to `dest`
Working	Copies characters from `src` to `dest` one by one, including the final `'\0'`

```c
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Programming";
    char dest[20];
    strcpy(dest, src);
    printf("Copied string: %s", dest);
    return 0;
}
```

OUTPUT

```
Copied string: Programming
```

3.2.1 Diagram — String Copy Process

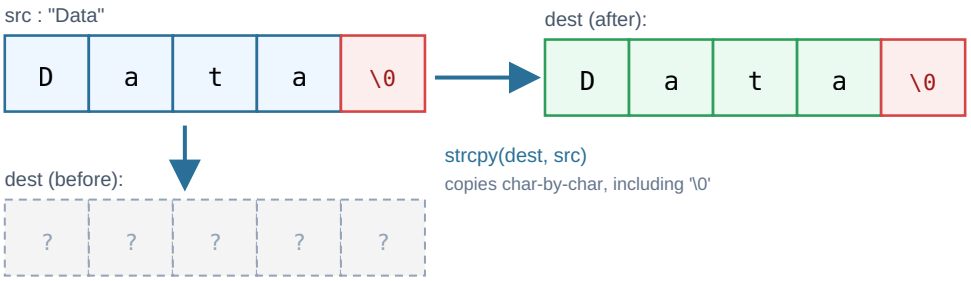


Fig 3.2 — strcpy() copies each character (including '\0') from src into dest, position by position.

Advantages

Simple, fast, widely supported; directly assigns one string's value to another (unlike arrays, which cannot use '=').

Limitations

No bounds checking — if `dest` is smaller than `src`, it causes **buffer overflow**. Prefer `strncpy()` when size is uncertain.

3.3 strcat() — String Concatenation

`strcat()` Appends (joins) one string to the end of another

```
SYNTAX
char *strcat(char *dest, const char *src);
```

Item	Detail
Parameters	`dest` — string to which `src` is appended; must have enough spare space
Return value	Pointer to `dest`
Working	Finds the `'\0'` of `dest`, then copies characters of `src` starting at that position, adding a new `'\0'` at the end

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str1[20] = "Hello, ";
    char str2[] = "World!";
    strcat(str1, str2);
```

```
    printf("%s", str1);
    return 0;
}
```

OUTPUT

```
Hello, World!
```

3.3.1 Diagram — String Concatenation Process

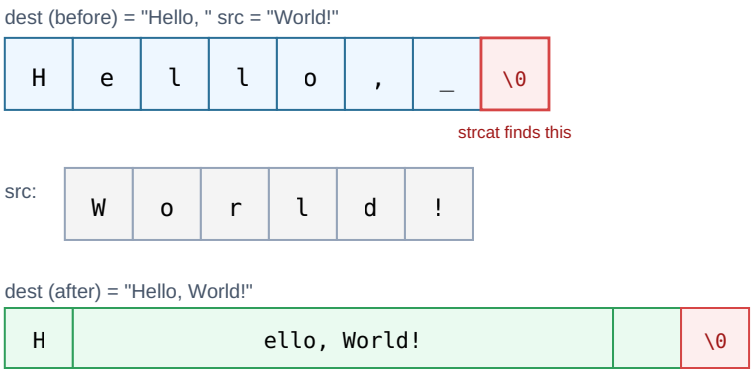


Fig 3.3 — strcat() locates the '\0' in dest and overwrites from there with src, appending a fresh '\0'.

3.4 strcmp() — String Comparison

strcmp() Compares two strings lexicographically (character by character)

SYNTAX

```
int strcmp(const char *str1, const char *str2);
```

Return value	Meaning
0	Both strings are identical
< 0 (negative)	`str1` is lexicographically less than `str2` (comes first alphabetically)
> 0 (positive)	`str1` is lexicographically greater than `str2`

Working mechanism

strcmp() compares corresponding ASCII values one pair at a time. The first mismatching pair decides the result: the difference (positive/negative/zero) reflects `str1[i] - str2[i]` at that position.

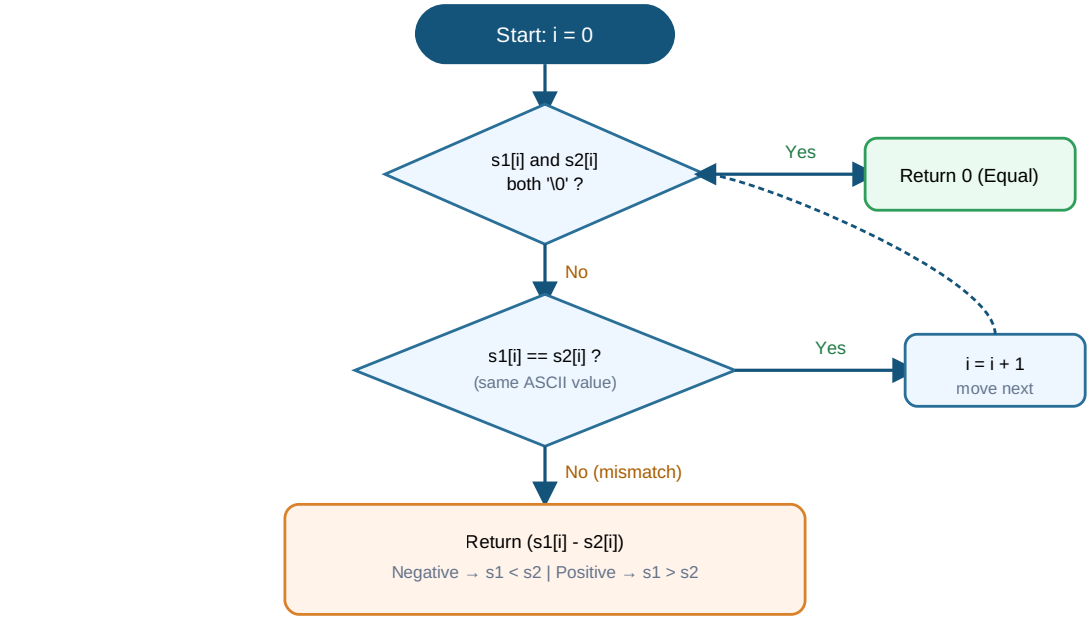
```c
#include <stdio.h>
#include <string.h>

int main() {
    char a[] = "apple", b[] = "apply";
    int result = strcmp(a, b);
    if (result == 0)
        printf("Strings are equal");
    else if (result < 0)
        printf("apple comes before apply");
    else
        printf("apple comes after apply");
    return 0;
}
```

OUTPUT

```
apple comes before apply
```

3.4.1 Flowchart — strcmp() Comparison Process



Loop continues while characters match; stops at first mismatch or when both reach '\0'.

Fig 3.4 — strcmp() compares characters left to right until a mismatch or both '\0' are reached.

3.5 Other Common String Functions

Function	Syntax	Purpose
`strncpy()`		

	`char *strncpy(char *dest, const char *src, size_t n);`	Copies at most `n` characters from `src` to `dest` — safer, bounded version of `strcpy()`
`strncat()`	`char *strncat(char *dest, const char *src, size_t n);`	Appends at most `n` characters from `src` to `dest` — bounded version of `strcat()`
`strncmp()`	`int strncmp(const char *s1, const char *s2, size_t n);`	Compares only the first `n` characters of the two strings

> **Exam Tip**
>
> Whenever a question mentions "buffer overflow" or "safe string handling", the expected answer usually involves replacing `strcpy/strcat/strcmp` with their `strn`-prefixed, bounded counterparts.

Chapter 3 Summary

Function	Purpose	Returns
`strlen(s)`	Length of string	`size_t` count
`strcpy(d,s)`	Copy s into d	pointer to d
`strcat(d,s)`	Append s to d	pointer to d
`strcmp(a,b)`	Compare a and b	0 / <0 / >0

CH 4 Similarity Between String and 1-D Array of Char

4.1 The Relationship

In C, a string **is** a one-dimensional array of type `char`. There is no independent string object at the language level; the compiler treats `char name[20]` exactly like any other array, except that library functions and literal syntax give it "string" behaviour through the null-terminator convention.

> **Key idea**
> Every string is a character array, but not every character array is a valid "string" — a char array only behaves as a string if it contains a `'\0'` somewhere within its bounds.

Example 1 — A valid string

```c
char name[20] = "Ravi";
// Stored as: R a v i \0 ? ? ? ... (20 bytes total)
printf("%s", name);   // prints "Ravi" — safe, because '\0' marks the end
```

```
OUTPUT
```

```
Ravi
```

Here `name` is a character array **and** a valid string, because the compiler automatically placed `'\0'` right after the last letter.

Example 2 — A char array that is NOT a valid string

```c
char grades[3] = {'A', 'B', 'C'};   // no '\0' anywhere
printf("%s", grades);   // UNSAFE — keeps reading past index 2
```

```
OUTPUT
```

```
ABC%@#&!... (garbage — undefined behaviour)
```

`grades` is a perfectly normal 1-D character array, but it is **not** a valid string — there is no `'\0'` to tell `printf("%s")` where to stop, so it keeps reading whatever bytes happen to follow in memory.

4.2 Similarities Between Strings and 1-D Character Arrays

- Both are declared the same way: `char arr[SIZE];`
- Both store elements of type `char` in contiguous memory locations.
- Both allow element access using index notation: `arr[i]`.
- The array name in both cases decays to a pointer to the first element (`char *`) when passed to a function.
- Both are fixed in size once declared (in the case of static arrays).

4.3 Differences Between a "Plain" Character Array and a String

Aspect	Plain Character Array	String (Special Char Array)
Terminator	Not required	Must end with `'\0'`
Purpose	General storage of characters (e.g. a grade list)	Represents readable text data
Library support	`strlen()`, `strcpy()` etc. not meaningful	Fully supported by `<string.h>` functions
Printing with `%s`	Unsafe / undefined if no `'\0'`	Safe — printing stops at `'\0'`
Example	`char grades[3] = {'A','B','C'};`	`char grades[4] = "ABC";` → `{'A','B','C','\0'}`

4.4 Storage Mechanism

When you write `char city[10] = "Delhi";`, the compiler:

1. Reserves 10 contiguous bytes of memory labelled `city[0]` through `city[9]`.
2. Copies the characters `D, e, l, h, i` into `city[0..4]`.
3. Places `'\0'` automatically at `city[5]`.
4. Leaves `city[6]` to `city[9]` uninitialised (garbage values), reserved for future use.

4.5 Memory Representation — Example

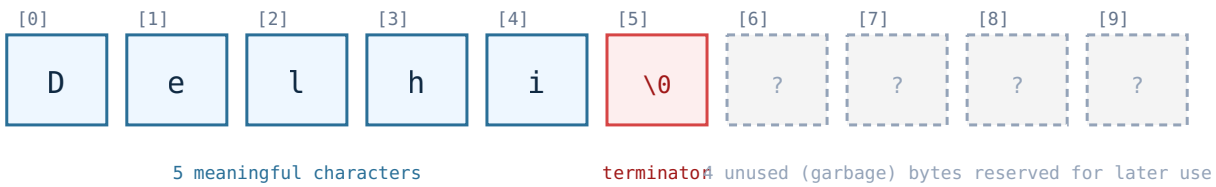


Fig 4.1 — char city[10] = "Delhi"; only 6 of 10 bytes are meaningful (5 letters + null terminator).

4.6 Worked Example

```c
#include <stdio.h>

int main() {
    char fruit[6] = "Mango";   // array of 6 chars
    int i;
    for (i = 0; fruit[i] != '\0'; i++)
        printf("%c", fruit[i]);
    return 0;
}
```

OUTPUT

```
Mango
```

This program treats `fruit` purely as an **indexable character array**, looping until the null terminator is reached — demonstrating that "string handling" is really just array traversal guided by `'\0'`.

4.7 Comparison Table: String vs 1-D Char Array (Consolidated)

Feature	1-D Character Array	String
Data type used	`char`	`char` (same)
Underlying structure	Array of characters	Array of characters (identical)
End marker	None required	`'\0'` mandatory
Indexing	Yes — `arr[i]`	Yes — `str[i]`
Direct assignment (`a = b`)	Not allowed	Not allowed (use `strcpy`)
Comparison (`a == b`)	Compares addresses, not content	Same — use `strcmp`
Standard functions	Not typically applicable	`strlen, strcpy, strcat, strcmp,` …
Printing with `%s`	Only safe if null-terminated	Always safe (by definition)

Chapter 4 Summary

- A string is a specialised, null-terminated one-dimensional character array.
- Structurally they are identical in memory; the difference is purely conventional (presence of `'\0'`).
- All string library functions rely on the null terminator to know where to stop.

CH 5 Programming Examples

5.1 Finding String Length (Without strlen)

```c
#include <stdio.h>

int main() {
    char str[] = "Exam";
    int count = 0;
    while (str[count] != '\0')
        count++;
    printf("Length = %d", count);
    return 0;
}
```

OUTPUT

```
Length = 4
```

Exam favourite: Writing a manual strlen()-equivalent using a while/for loop is one of the most frequently asked programs in university and TET/recruitment exams.

5.2 Copying a String (Without strcpy)

```c
#include <stdio.h>

int main() {
    char src[] = "Kernel", dest[20];
    int i;
    for (i = 0; src[i] != '\0'; i++)
        dest[i] = src[i];
    dest[i] = '\0';         // manually terminate
    printf("%s", dest);
    return 0;
}
```

OUTPUT

```
Kernel
```

5.3 Concatenating Two Strings (Using strcat)

```c
#include <stdio.h>
#include <string.h>

int main() {
    char a[30] = "Data", b[] = "Structures";
    strcat(a, b);
    printf("%s", a);
    return 0;
}
```

OUTPUT

```
DataStructures
```

5.4 Comparing Two Strings (Using strcmp)

```c
#include <stdio.h>
#include <string.h>

int main() {
    char p[20], q[20];
    printf("Enter two words: ");
    scanf("%s %s", p, q);
    if (strcmp(p, q) == 0)
        printf("Equal");
    else
        printf("Not Equal");
    return 0;
}
```

SAMPLE RUN

```
Enter two words: cat cat
Equal
```

5.5 Checking if a String is a Palindrome

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[20] = "madam";
    int i, len = strlen(s), flag = 1;
    for (i = 0; i < len / 2; i++)
        if (s[i] != s[len - 1 - i]) { flag = 0; break; }
    printf(flag ? "Palindrome" : "Not a Palindrome");
    return 0;
}
```

OUTPUT

```
Palindrome
```

5.6 Reversing a String

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[20] = "Coding";
    int len = strlen(s);
    for (int i = len - 1; i >= 0; i--)
        printf("%c", s[i]);
    return 0;
}
```

OUTPUT

```
gnidoC
```

Chapter 5 Summary

These six programs cover the core exam patterns: manual re-implementation of library functions (tests understanding of the null terminator) and practical use of `<string.h>` functions in real tasks such as palindrome checking and reversal.

CH 6 Common Errors and Best Practices

6.1 Missing Null Terminator

Error

`char s[5] = {'H','e','l','l','o'};` — no '\0' is present. Using `printf("%s", s)` or `strlen(s)` on this array reads past its end into unrelated memory, producing garbage characters or a crash.

Fix

Always reserve one extra byte and set it explicitly: `char s[6] = {'H','e','l','l','o','\0'};`

6.2 Buffer Overflow

Error

`char dest[5]; strcpy(dest, "Hello World");` — the source string is longer than the destination array, overwriting adjacent memory (buffer overflow), which can corrupt other variables or crash the program.

Fix

Use bounded functions: `strncpy(dest, src, 4); dest[4] = '\0';` and always ensure the destination is large enough for source length + 1.

6.3 Incorrect String Handling

Mistake	Why it's wrong	Correct approach
`if (str1 == str2)`	Compares memory addresses, not content	Use `strcmp(str1, str2) == 0`
`str1 = str2;`	Arrays cannot be assigned directly	Use `strcpy(str1, str2);`
Using `gets()`	No bounds checking; deprecated/removed in modern C standards	Use `fgets(str, size, stdin);`
Forgetting `<string.h>`	Compiler may not recognise `strlen`, etc.	Always `#include <string.h>`
Off-by-one array size	Not accounting for `'\0'`	Declare size = max characters + 1

6.4 Safe Programming Practices

• Always allocate at least one extra byte for the null terminator.

- Prefer `strncpy()`, `strncat()`, `strncmp()` over their unbounded versions when the input size is not guaranteed.
- Validate input length before copying or concatenating.
- Use `fgets()` instead of `gets()` / `scanf("%s")` when reading untrusted or variable-length input.
- Initialise character arrays to avoid garbage values, e.g. `char buf[20] = {0};`

Chapter 6 Summary

Most string bugs trace back to two root causes: a **missing/incorrect null terminator** and **insufficient destination size**. Bounded (`strn...`) functions and careful size arithmetic prevent almost all of them.

CH 7 Quick Revision Section

7.1 Important Definitions

String: A one-dimensional array of characters terminated by the null character `'\0'`.

Null character: `'\0'`, ASCII 0, marks the logical end of a string.

String literal: Text in double quotes (e.g. `"Hi"`), stored with an automatic `'\0'`.

Buffer overflow: Writing more characters into an array than its declared size allows.

7.2 Syntax Summary Table

Function	Syntax
strlen	`size_t strlen(const char *str);`
strcpy	`char *strcpy(char *dest, const char *src);`
strcat	`char *strcat(char *dest, const char *src);`
strcmp	`int strcmp(const char *s1, const char *s2);`
strncpy	`char *strncpy(char *dest, const char *src, size_t n);`
strncat	`char *strncat(char *dest, const char *src, size_t n);`
strncmp	`int strncmp(const char *s1, const char *s2, size_t n);`

7.3 Function Comparison Table

Function	Purpose	Modifies argument?	Return type	Bounded version
strlen	Measure length	No	size_t	—
strcpy	Copy string	Yes (dest)	char*	strncpy
strcat	Join strings	Yes (dest)	char*	strncat
strcmp	Compare strings	No	int	strncmp

7.4 String vs 1-D Character Array — One-Look Chart

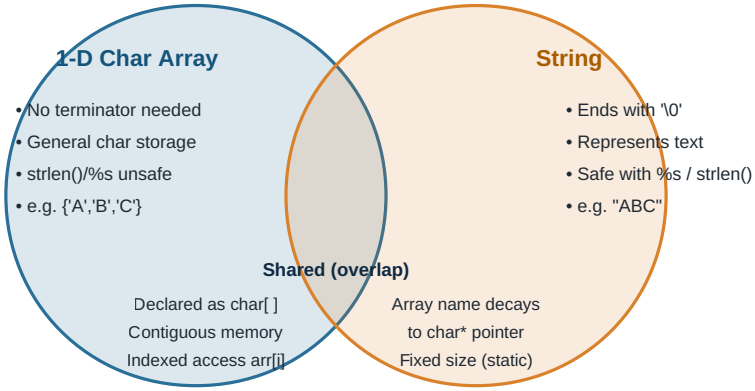


Fig 7.1 — Both share identical storage; the null terminator is the sole distinguishing feature of a "string".

7.5 Frequently Asked Exam Questions

Q1. Why is a string called a "null-terminated character array"?

Because it is stored exactly like a char array, but its end is explicitly marked by '\0', which all string functions rely on.

Q2. What happens if you forget the null terminator?

Functions like strlen/printf("%s") read beyond the array into unrelated memory — undefined behaviour.

Q3. Differentiate strcpy() and strncpy().

strcpy() copies the entire source regardless of destination size (risk of overflow); strncpy() copies at most n characters, giving control over how much is written.

Q4. What does strcmp() return when strings are equal?

0 (zero).

Q5. Can you compare two strings using the == operator?

No — it compares pointer addresses, not contents; use strcmp() instead.

Q6. Is every character array a valid string in C?

No — only if it contains a '\0' within its bounds; otherwise it is just a plain char array.

7.6 One-Page Revision Notes

Everything in one glance

- String = char array + `'\0'` terminator.
- Declare with size = (max chars + 1) to leave room for `'\0'`.

- `strlen` → counts characters before `'\0'` (excludes it).
- `strcpy(dest, src)` → overwrites dest completely, including terminator.
- `strcat(dest, src)` → finds dest's `'\0'`, appends src there.
- `strcmp(a, b)` → 0 if equal, negative if a < b, positive if a > b (ASCII-wise).
- Use `strncpy/strncat/strncmp` for bounded, safer operations.
- Never use `==` or `=` directly on strings — always use library functions.

7.7 Common Mistakes and Debugging Tips

Symptom	Likely Cause	Debugging Tip
Garbage characters printed	Missing '\0'	Print array length vs expected size; check every element
Program crashes on strcpy/strcat	Destination array too small	Verify destination size ≥ source length + 1
if(str1==str2) always false	Comparing addresses not content	Replace with strcmp() == 0
strcmp gives unexpected sign	Case sensitivity (ASCII 'A' ≠ 'a')	Convert both strings to same case before comparing if case-insensitive match is needed

Final Takeaway

A C string is nothing more than a one-dimensional character array with a null terminator as its defining feature. Master the four core functions — `strlen, strcpy, strcat, strcmp` — and their bounded `strn...` counterparts, and you can confidently answer virtually any exam question on character strings in C.